Category C Assignment Problems

Category C · 5 Assignment problems selected for logic-building using arrays, loops, and conditionals only

A1. Product of Array Except Self

Scenario

A pricing engine needs, for every product in a bundle, the combined price of every OTHER product in that same bundle — computed for all products at once, without ever dividing by the current product's own price (some prices could legitimately be zero, e.g. a free promotional item, which would make division undefined).

Task

- Accept an integer array nums.
- Return an array answer where answer[i] is the product of every element in nums except nums[i] — without using division anywhere in your solution.
- Solve it in O(n) time using two passes: a forward pass accumulating the running product of everything to the left of each index, and a backward pass multiplying in the running product of everything to the right.

Suggested Method Signature(s)

```
int[] productExceptSelf(int[] nums)
```

Sample Input / Output

Input	Output
nums = [1, 2, 3, 4]	[24, 12, 8, 6]
nums = [-1, 1, 0, -3, 3]	[0, 0, 9, 0, 0] (a single zero forces every OTHER position to be 0 except the zero's own position)

Constraints

- 2 ≤ nums.length ≤ 10^5.
- The product of any prefix or suffix fits within a 32-bit integer.
- Division is not allowed anywhere in the solution, even to “cancel it out” for zero handling.

Concepts covered: *Prefix and suffix products, two-pass array traversal, handling zero values without dividing, O(n) time with O(1) extra space beyond the output array.*

A2. Maximum Subarray

Scenario

A trader has a full year of daily profit-or-loss figures, some positive, some negative, and wants to know the single best contiguous stretch of days to have been actively trading — the run of consecutive days whose combined total is the highest possible.

Task

- Accept an integer array nums, which may contain negative numbers.
- Find the contiguous subarray (containing at least one number) with the largest possible sum, and return that sum.
- Solve it using Kadane's algorithm: at each element, decide whether to extend the current running subarray or abandon it and start fresh from the current element, based on whichever gives a larger sum.

Suggested Method Signature(s)

```
int maxSubArray(int[] nums)
```

Sample Input / Output

Input	Output
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]	6 (the subarray [4, -1, 2, 1] sums to 6)
nums = [-3, -1, -2]	-1 (all negative -- the best you can do is the single largest value)

Constraints

- 1 ≤ nums.length ≤ 10^5.
- Solve it in O(n) time and O(1) extra space.
- Be ready to explain the O(n log n) divide-and-conquer alternative as a follow-up — interviewers often ask for it after Kadane's.

Concepts covered: *Kadane's algorithm, the "extend vs. restart" decision at each step, running-sum reset logic, handling an all-negative array correctly.*

A3. 3Sum

Scenario

A budgeting tool needs to find every distinct combination of exactly three transactions in a student's account history that cancel each other out exactly — summing to zero — without ever reporting the same combination of amounts twice, even if it could be picked out of the list in more than one way.

Task

- Accept an integer array nums.
- Return all unique triplets [nums[i], nums[j], nums[k]] (i, j, k all different positions) such that the three values sum to exactly 0.
- Sort the array first, then for each element, use two pointers moving inward from both ends of the remaining subarray to find pairs that complete the sum to zero — carefully skipping over duplicate values at every level to avoid reporting the same triplet more than once.

Suggested Method Signature(s)

```
int[][] threeSum(int[] nums)
```

Sample Input / Output

Input	Output
nums = [-1, 0, 1, 2, -1, -4]	[[-1, -1, 2], [-1, 0, 1]]
nums = [0, 0, 0]	[[0, 0, 0]] (only reported once, despite three identical values)

Constraints

- 3 ≤ nums.length ≤ 3000.
- Aim for O(n²) time — the sort costs O(n log n), and the two-pointer scan per element costs O(n), giving O(n²) overall.
- Duplicate handling is the single most common source of bugs in this problem — test an input with several repeated values before considering it done.

Concepts covered: *Sorting as a setup step, the two-pointer technique on a sorted array, systematic duplicate avoidance, converting a 2-sum idea into a 3-sum solution.*

A4. Subarray Sum Equals K

Scenario

A hostel warden reviewing a semester's daily attendance-change log (some days net positive, some negative, as students check in and out) wants to know how many different contiguous stretches of days had a net change of exactly k — across the whole log, not just one answer.

Task

- Accept an integer array nums (which may contain negative numbers) and an integer k.
- Return the total number of contiguous subarrays whose sum equals exactly k.
- Solve it using running prefix sums combined with a hash map: the sum of any subarray [i+1, j] equals prefixSum[j] − prefixSum[i], so at each position you need to know how many earlier prefix sums equal (currentSum − k).
- Think first about why a sliding-window approach breaks down here, given that the array can contain negative numbers.

Suggested Method Signature(s)

```
int subarraySum(int[] nums, int k)
```

Sample Input / Output

Input	Output
nums = [1, 1, 1], k = 2	2 (the subarrays [1,1] at positions 0-1 and 1-2 both sum to 2)
nums = [1, -1, 0], k = 0	3

Constraints

- 1 ≤ nums.length ≤ 2 × 10^4.
- nums[i] and k may both be negative.
- Solve it in O(n) time and O(n) space using a hash map of prefix-sum frequencies.

Concepts covered: *Prefix sums, hash map frequency counting, why sliding-window techniques require non-negative values to work correctly, careful initialization of the "empty prefix" base case.*

A5. Find Minimum in Rotated Sorted Array

Scenario

A circular duty roster was originally sorted by join date, then "rotated" at some unknown point when the office started the printed list from a different staff member instead of the very first one. Given only the resulting list, find the original earliest join date — without scanning every entry one by one.

Task

- Accept an integer array nums of unique elements, originally sorted in ascending order and then rotated at some unknown pivot.
- Return the minimum element in the array.
- Solve it using a modified binary search rather than a linear scan: at each step, compare the middle element to the rightmost element to decide which half of the array the minimum must be hiding in.

Suggested Method Signature(s)

```
int findMin(int[] nums)
```

Sample Input / Output

Input	Output
nums = [3, 4, 5, 1, 2]	1
nums = [4, 5, 6, 7, 0, 1, 2]	0
nums = [11, 13, 15, 17]	11 (no rotation actually occurred)

Constraints

- 1 ≤ nums.length ≤ 5000.
- All elements are distinct.
- Solve it in O(log n) time — a full linear scan will be correct but will not earn full marks.

Concepts covered: *Binary search adapted to a rotated (not fully sorted) array, deciding which half genuinely contains the answer, correctly handling the already-sorted (no-rotation) case as well.*